TECHNICAL REPORT

Machine Learning Project 2

Marwan Deif
13004051
Mazen Yasser
7000639

1.5.1 Data Understanding & Exploration

1. Findings from EDA

- Relationship & join keys: Beneficiary-level data (BeneID) links to claim tables (inpatient/outpatient). Provider is the modeling unit; Provider IDs appear in claim files and labels.

- Data quality: Several columns show missing values. Investigate high-missing columns and decide on imputation or removal. Duplicate/invalid Provider or BeneID rows should be checked.

- Beneficiary patterns: Age/gender distributions and counts of chronic conditions (if available) help profile patient mixes per provider.

- Claim-level patterns: Claim amount distributions are heavy-tailed (log-transform recommended). Temporal trends (monthly) may reveal spikes for certain providers.

- Fraud vs legitimate: Provider-level aggregated features (claims_count, mean claim amount) show different distributions between labeled fraudulent and non-fraudulent providers — visualize via boxplots and histograms.

- Aggregation strategy: We created provider-level features including counts, sums, means, medians, stds, unique beneficiary counts, and claims-per-beneficiary ratios. Saved as `data/final_provider_dataset_from_EDA.csv`.

- Core plots generated: class distribution, claim amount trends, provider-level summaries, correlation heatmap; geographic/temporal plots attempted if columns available.

---

2. Next steps / recommendations

1. Clean and impute missing values (domain-specific choices: zero, median, or model-based imputation).
2. Feature engineering: create temporal deltas, provider peer-group statistics, network features (shared patients), and coding-frequency features (rare CPT codes).
3. For modeling, handle class imbalance with class weights, SMOTE/oversampling, or cost-sensitive learning (discussed in the modeling notebook).
4. Run feature selection, then modeling with cross-validation and hyperparameter tuning.
5. Perform detailed error analysis on false positives/negatives to refine features.

1.5.2 Class Imbalance Strategy — Justification

The dataset is highly imbalanced, with fraudulent providers representing a small minority.

To address this imbalance, two approaches were implemented:

1. Class Weighting

- Used in Logistic Regression.
- Automatically penalizes misclassifying the minority class more heavily.
- Advantage: preserves original data distribution.
- Good for interpretability and transparency.

2. SMOTE Oversampling

- Used in Random Forest model.
- Generates synthetic minority samples to balance the training set.
- Advantage: improves recall and PR-AUC significantly.
- Trade-off: may introduce noise and reduce model interpretability.

Metrics Used

Because accuracy is misleading in imbalanced datasets, we prioritized:

- Precision (how many predicted frauds are correct)
- Recall (ability to detect fraud)
- F1-score (balance between precision & recall)
- PR-AUC (best overall metric for imbalance)
- ROC-AUC (secondary)

Trade-offs

- Class-weighted models offer clearer interpretability.
- SMOTE + Random Forest provides superior fraud detection power but less interpretability.
- Recommended: RF + SMOTE as primary fraud detection model, with logistic regression as an interpretable fallback.

1.5.3-1.5.4 Model selection justification & trade-offs

Algorithms evaluated: Decision Tree, Random Forest, Gradient Boosting, Logistic Regression, SVM.

Considerations we used:

- Interpretability: Logistic Regression and Decision Tree are most interpretable (coefficients, simple rules). Tree ensembles are less transparent but allow feature importance and SHAP explanations.

- Computational feasibility: Logistic Regression and Decision Trees are fast. SVM with RBF can be slow for many features. Gradient Boosting (scikit-learn) and Random Forest are balanced in speed vs performance; XGBoost/LightGBM would be faster and often more accurate if available.

- Robustness to imbalance: Models trained with `class_weight='balanced'` and/or with SMOTE were tested. Tree ensembles and class-weighted logistic regression showed stable behavior; SMOTE improved recall at the expense of potential overfitting.

- Suitability for mixed data: Tree-based models handle mixed numeric/categorical data naturally. Logistic Regression and SVM require encoding and careful scaling.

Primary model recommendation:

Based on PR-AUC and F1 on the test set (see results saved to `model_comparison_final_results.csv`), **Gradient Boosting (tuned)** is recommended as the primary model due to:

- Strong predictive performance on imbalanced data (high PR-AUC).

- Ability to capture nonlinear interactions common in fraud patterns.

- Regularization controls (learning_rate, max_depth, subsample) to limit overfitting.

- Explainability via feature importances and SHAP values for per-case explanations to investigators.

Secondary / comparison models:

- Random Forest (tuned) — robust baseline, often similar performance, faster to explain via feature importance.

- Logistic Regression (class_weight) — interpretable fallback for regulatory transparency or triage scoring.

Trade-offs:

- Gradient Boosting offers top predictive power but less inherent interpretability. Use SHAP for local explanations when presenting flagged providers to investigators.

- Logistic Regression is transparent but may miss complex fraud patterns (lower recall).

- SMOTE improved recall for some models but can introduce synthetic-sample bias — recommended only after careful validation and possibly combined with ensemble techniques.

Operational recommendation:*

- Deploy tuned Gradient Boosting as the scoring model.

- Build an explanation pipeline (SHAP) that returns top contributing features per flagged provider.

- Use Logistic Regression as a parallel explainability check or for communication with auditors/regulators.

- Monitor drift and re-evaluate monthly with new labeled cases.

1.6 Evaluation Metric

Cost-Based Analysis

We examine the real-world implications of false positives (FPs) and false negatives (FNs):

- **False Positive (FP)** — a legitimate provider flagged as fraudulent. This leads to investigation costs and potential reputational harm. Example costs include administrative investigation time, audits, and provider disruption.
- **False Negative (FN)** — a fraudulent provider missed by the model. This leads to financial loss (continued fraud payments) and reduced trust in detection systems.

Quantitative example (illustrative):

- Suppose average investigation cost per FP ≈ $5,000.
- Suppose average loss per FN (undetected fraud) ≈ $50,000.

Using these assumptions:

- Total FP cost = FP_count * 5,000
- Total FN cost = FN_count * 50,000

This shows that a single FN can cost as much as ten FPs — therefore, the operational policy might prefer higher recall (catch more fraud) even at the expense of more FPs, up to a tolerable threshold.

Recommendation: choose model threshold and operating point by optimizing expected cost (or expected utility) rather than overall accuracy. A threshold sweep and expected-cost curve are recommended for production deployment.

Error Analysis: Understanding False Positives and False Negatives

After identifying 3 false positive (FP) and 3 false negative (FN) cases, we examine the provider-level features to understand why the model made these incorrect predictions and what patterns contributed to the errors.

`

1. **False Positives (Legitimate providers incorrectly flagged as fraud)**

These providers received high fraud probabilities despite being legitimate. From inspection of feature tables, common contributing patterns include:

Key Reasons for FPs

- Unusual claim frequency or sudden spikes

    - Some legitimate providers have temporarily high transaction counts (e.g., seasonal demand or new service launches), which the model incorrectly interprets as suspicious.
- High similarity to typical fraudulent patterns
    - A few legitimate providers show:
        - unusually high "average claim amount"
        - repeated common procedure combinations
        - These patterns resemble known fraud signatures in the training data.
- Provider type imbalance
    - Some provider categories (e.g., small clinics) may be overrepresented in fraud cases, biasing the classifier.

2. **False Negatives (Fraudulent providers missed by the model)**

False negatives typically occur when fraudulent behavior does not strongly match the dominant patterns seen in the training dataset.

Key Reasons for FNs

- Fraud profiles that appear "normal"
    - Some fraudulent providers closely mimic regular behavior:
        - moderate claim amounts
        - average claim frequencies
        - no extreme outliers

These subtle forms of fraud evade detection because they do not align with the more obvious patterns the model learned.

- Insufficient representation of similar fraud types in training data
    - If the training set underrepresents certain fraud schemes, the classifier struggles to detect those variants.
- Features not discriminative enough
    - The model might be missing:

- temporal fraud signals
- network-level behaviors
- unusual provider-patient interactions

These features often improve fraud detection but may not exist in the current dataset.

3. **Recommendations for Model Refinement**

Add More Discriminative Features:-

- Time-based features
    - Claim trends over months
    - Sudden spikes or anomalie
- Provider network features
    - Shared patients across providers
    - Unusual referral loops
- Behavioral patterns
    - Claim repetition frequency
    - Rare combinations of procedures

These features are known to significantly improve fraud detection accuracy.

How Overfitting Was Prevented:

1. Data Partitioning

We used a three-way split:

- 60% Training
- 20% Validation
- 20% Test

This structure prevents information leakage because:

- The training set is used to fit the model
- The validation set is used for model selection and hyperparameter tuning
- The test set is used only at the end for final unbiased evaluation

We also used stratified splitting to preserve the fraud ratio, which is important due to severe class imbalance.

2. Regularization

We applied L2 regularization in Logistic Regression:

- Shrinks large coefficients
- Reduces sensitivity to noise
- Helps the model generalize better
- The parameter C = 0.5 increases regularization strength

Regularization is essential when using many engineered features derived from claims data.